

What is .NET? Introduction and overview

Article • 11/15/2023

.NET is a [free](#) , cross-platform, [open-source developer platform](#) for building many kinds of applications. .NET is built on a [high-performance runtime](#) that is used in production by many [high-scale apps](#) .

Cloud apps

- [Cloud native apps](#)
- [.NET Aspire](#)
- [Console apps](#)
- [Serverless functions in the cloud](#)
- [Web apps, web APIs, and microservices](#)

Cross-platform client apps

- [Desktop apps](#)
- [Games](#)
- [Mobile apps](#)

Windows apps

- [Windows Desktop apps](#)
 - [Windows Forms](#)
 - [Windows WPF](#)
 - [Universal Windows Platform \(UWP\)](#)
- [Windows services](#)

Other app types

- [Machine learning](#)
- [Internet of Things \(IoT\)](#)

Features

.NET features allow developers to productively write reliable and performant code.

- [Asynchronous code](#)
- [Attributes](#)

- [Reflection](#)
- [Code analyzers](#)
- [Delegates and lambdas](#)
- [Events](#)
- [Exceptions](#)
- [Garbage collection](#)
- [Generic types](#)
- [LINQ \(Language Integrated Query\)](#).
- [Parallel programming and Managed threading](#)
- [Type inference - C#, F#, Visual Basic](#).
- [Type system](#)
- [Unsafe code](#)

Using .NET

.NET apps and libraries are built from source code and a project file, using the [.NET CLI](#) or an Integrated Development Environment (IDE) like [Visual Studio](#) .

The following example is a minimal .NET app:

Project file:

XML

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
  </PropertyGroup>
</Project>
```

Source code:

C#

```
Console.WriteLine("Hello, World!");
```

The app can be built and run with the [.NET CLI](#):

.NET CLI

```
dotnet run  
Hello, World!
```

Binary distributions

- [.NET SDK](#): Set of tools, libraries, and runtimes for development, building, and testing apps.
- [.NET Runtimes](#) : Set of runtimes and libraries, for running apps.

You can download .NET from:

- [The Microsoft download site](#) .
- [Containers](#) .
- [Linux package managers](#).

Free and open source

.NET is free, open source, and is a [.NET Foundation](#) project. .NET is maintained by Microsoft and the community on GitHub in [several repositories](#) .

.NET source and binaries are licensed with the [MIT license](#) . Additional [licenses apply on Windows](#) for binary distributions.

Support

[Microsoft supports .NET](#) on Android, Apple, Linux, and Windows operating systems. It can be used on Arm64, x64, and x86 architectures. It's also supported in emulated environments, like [macOS Rosetta 2](#) .

New versions of .NET are released annually in November. .NET releases in odd-numbered years are Long-Term Support (LTS) releases and are supported for three years. Releases in even-numbered years are Standard-Term Support (STS) releases and are supported for 18 months. The quality level, breaking change policies, and all other aspects of the releases are the same. For more information, see [Releases and support](#).

The .NET Team at Microsoft works collaboratively with other organizations to distribute and support .NET in various ways.

[Red Hat supports .NET](#) on Red Hat Enterprise Linux (RHEL).

[Samsung supports .NET](#) on Tizen platforms.

Runtime

The [Common Language Runtime \(CLR\)](#) is the foundation all .NET apps are built on. The [fundamental features of the runtime](#) are:

- Garbage collection.
- Memory safety and type safety.
- High level support for programming languages.
- Cross-platform design.

.NET is sometimes called a "managed code" runtime. It's called *managed* primarily because it uses a garbage collector for memory management and because it enforces type and memory safety. The CLR virtualizes (or abstracts) various operating system and hardware concepts, such as memory, threads, and exceptions.

The CLR was designed to be a cross-platform runtime from its inception. It has been ported to multiple operating systems and architectures. Cross-platform .NET code typically does not need to be recompiled to run in new environments. Instead, you just need to install a different runtime to run your app.

The runtime exposes various [diagnostics](#) services and APIs for debuggers, [dumps](#) and [tracing](#) tools, and [observability](#). The observability implementation is primarily [built around OpenTelemetry](#), enabling [flexible application monitoring](#) and site reliability engineering (SRE).

The runtime offers low-level C-style interop functionality, via a combination of [P/Invoke](#), value types, and the ability to [blit](#) values across the native/managed-code boundary.

Languages

The runtime is designed to support multiple programming languages. C#, F#, and Visual Basic languages are supported by Microsoft and are designed in collaboration with the community.

- [C#](#) is a modern, object-oriented, and type-safe programming language. It has its roots in the C family of languages and will be immediately familiar to C, C++, Java,

and JavaScript programmers.

- [F#](#) is an interoperable programming language for writing succinct, robust, and performant code. F# programming is data-oriented, where code involves transforming data with functions.
- [Visual Basic](#) uses a more verbose syntax that is closer to ordinary human language. It can be an easier language to learn for people new to programming.

Compilation

.NET apps (as written in a high-level language like C#) are compiled into an [Intermediate Language \(IL\)](#). IL is a compact code format that can be supported on any operating system or architecture. Most .NET apps use APIs that are supported in multiple environments, requiring only the .NET runtime to run.

IL needs to be compiled to native code to execute on a CPU, for example, Arm64 or x64. .NET supports both Ahead-Of-Time (AOT) and Just-In-Time (JIT) compilation models.

- On Android, macOS, and Linux, JIT compilation is the default, and AOT is optional (for example, with [ReadyToRun](#)).
- On [iOS](#), AOT is mandatory (except when running in the simulator).
- In WebAssembly (Wasm) environments, AOT is mandatory.

The advantage of the JIT is that it can compile an app (unmodified) to the CPU instructions and calling conventions in a given environment, per the underlying operating system and hardware. It can also compile code at higher or lower levels of quality to enable better startup and steady-state throughput performance.

The advantage of AOT is that it provides the best app startup and can (in some cases) result in smaller deployments. The primary downside is that binaries must be built for each separate deployment target (the same as any other native code). AOT code is not compatible with some reflection patterns.

Runtime libraries

.NET has a comprehensive standard set of class libraries. These libraries provide implementations for many general-purpose and workload-specific types and utility functionality.

Here are some examples of types defined in the .NET runtime libraries:

- Every .NET type derives from the [System.Object](#) type.
- Primitive value types, such as [System.Boolean](#) and [System.Int32](#).
- Collections, such as [System.Collections.Generic.List<T>](#) and [System.Collections.Generic.Dictionary<TKey,TValue>](#).
- Data types, such as [System.Data.DataSet](#) and [System.Data.DataTable](#).
- Network utility types, such as [System.Net.Http.HttpClient](#).
- File and stream I/O utility types, such as [System.IO.FileStream](#) and [System.IO.TextWriter](#).
- [Serialization](#) utility types, such as [System.Text.Json.JsonSerializer](#) and [System.Xml.Serialization.XmlSerializer](#).
- High-performance types, such as [System.Span<T>](#), [System.Numerics.Vector](#), and [Pipelines](#).

For more information, see the [Runtime libraries overview](#).

NuGet Package Manager

[NuGet](#) is the package manager for .NET. It enables developers to share compiled binaries with each other. [NuGet.org](#) offers many [popular packages](#) from the community.

Tools

The [.NET SDK](#) is a set of libraries and [tools](#) for developing and running .NET applications. It includes the [MSBuild](#) build engine, the [Roslyn](#) (C# and Visual Basic) compiler, and the [F#](#) compiler. Most commands are run by using the [dotnet command](#). The CLI tools can be used for local development and continuous integration.

The [Visual Studio](#) family of IDEs offer excellent support for .NET and the C#, F#, and Visual Basic languages.

[GitHub Codespaces](#) and [GitHub security features](#) support .NET.

Notebooks

.NET Interactive is a group of CLI tools and APIs that enable users to create interactive experiences across the web, markdown, and notebooks.

For more information, see the following resources:

- [.NET In-Browser Tutorial](#)
- [Using .NET notebooks with Jupyter on your machine](#)
- [.NET Interactive documentation](#)

CI/CD

MSBuild and the .NET CLI can be used with various continuous integration tools and environments, such as:

- [GitHub Actions](#)
- [GitHub Actions and .NET](#)
- [Azure DevOps](#)
- [CAKE for C#](#)
- [FAKE for F#](#)

For more information, see [Use the .NET SDK in Continuous Integration \(CI\) environments](#).

Deployment models

.NET apps can be [published in two different modes](#):

- *Self-contained* apps include the .NET runtime and dependent libraries. They can be [single-file](#) or multi-file. Users of the application can run it on a machine that doesn't have the .NET runtime installed. Self-contained apps always target a single operating system and architecture configuration.
- *Framework-dependent* apps require a compatible version of the .NET runtime, typically installed globally. Framework-dependent apps can be published for a single operating system and architecture configuration or as "portable," targeting all supported configurations.

.NET apps are launched with a native executable, by default. The executable is both operating-system and architecture-specific. Apps can also be launched with the [dotnet command](#).

Apps can be [deployed in containers](#). Microsoft provides [container images](#) for various target environments.

.NET history

In 2002, Microsoft released [.NET Framework](#), a development platform for creating Windows apps. Today .NET Framework is at version 4.8 and remains [fully supported by Microsoft](#) .

In 2014, Microsoft introduced .NET Core as a cross-platform, open-source successor to .NET Framework. This new [implementation of .NET](#) kept the name .NET Core through version 3.1. The next version after .NET Core 3.1 dropped the "Core" part of the name and was named .NET 5.

New .NET versions continue to be released annually, each a major version number higher. They include significant new features and often enable new scenarios.

.NET ecosystem

There are multiple variants of .NET, each supporting a different type of app. The reason for multiple variants is part historical, part technical.

.NET implementations (historical order):

- **.NET Framework** -- It provides access to the broad capabilities of Windows and Windows Server. Also extensively used for Windows-based cloud computing. The original .NET.
- **Mono** -- A cross-platform implementation of .NET Framework. The original community and open source .NET. Used for Android, iOS, and Wasm apps.
- **.NET (Core)** -- A cross-platform and open source implementation of .NET, rethought for the cloud age while remaining significantly compatible with .NET Framework. Used for Linux, macOS, and Windows apps.

Next steps

- [Choose a .NET tutorial](#)
- [Try .NET in your browser](#)
- [Take a tour of C#](#)
- [Take a tour of F#](#)

Collaborate with us on GitHub

The source for this content can be found on GitHub, where you can also create and review issues and pull requests. For more information, see [our contributor guide](#).



.NET feedback

.NET is an open source project. Select a link to provide feedback:

 [Open a documentation issue](#)

 [Provide product feedback](#)